

Hadoop (EXERCISE)

Friday, June 20, 2025 11:34 AM

2.1 What is Hadoop?

1. List three core capabilities Hadoop provides that make it suitable for “big data” workloads.
2. Explain how Hadoop achieves fault tolerance at the storage layer.
3. Why is commodity hardware key to Hadoop’s cost model?
4. Give one real-world scenario where Hadoop would be preferred over a single-machine data warehouse.
5. What trade-offs come with Hadoop’s batch-oriented design?

2.2 Key Components of the Hadoop Ecosystem

1. Match each layer (Storage, Processing, Coordination, Query & Tools) to its primary Hadoop component.
2. Briefly describe the role of YARN in a mixed-workload cluster.
3. Contrast Hive versus Pig in terms of user audience and language style.
4. Name two ingestion tools in this ecosystem and the sources they typically serve.
5. How does HBase differ from HDFS as a storage solution?

2.3 HDFS – Hadoop Distributed File System

1. If you upload a 500 MB file (default 128 MB blocks, replication factor 3), how many blocks are created and what’s the total raw storage consumed?
2. Describe what happens to clients’ read requests when one DataNode holding a needed block is down.
3. Why is the default block size much larger than on typical local filesystems?
4. How does rack awareness reduce both failure risk and network congestion?
5. What metadata burden arises from storing millions of small files, and how can you mitigate it?

2.4 MapReduce – Processing Engine

1. Outline the sequence of steps in a MapReduce job from client submission through final output.
2. For the classic word-count example, what key–value pairs does the Mapper emit, and what does the Reducer aggregate?
3. Explain how a Combiner can reduce network traffic. When shouldn’t you use one?
4. Identify two limitations of MapReduce that led to newer engines like Spark.
5. How would you decide on the number of reducers for a given job?

2.5 YARN – Yet Another Resource Negotiator

1. Differentiate the responsibilities of the ResourceManager versus a NodeManager.
2. Explain speculative execution: when it kicks in and how it speeds up jobs.
3. How does YARN enable multiple frameworks (MapReduce, Spark, Hive) to coexist?
4. What happens if YARN’s scheduler cannot find enough free resources for an incoming application?
5. Suggest two configuration tweaks to prevent container-launch failures.

2.6 Hive & Pig

1. Write a simple HiveQL query to count rows in a table users.
2. Show a Pig Latin snippet that loads data.csv, filters for age > 30, and stores the result.
3. Compare how Hive and Pig translate user code into underlying MapReduce or Tez jobs.
4. In which use cases would you choose Pig over Hive, and why?
5. How does Hive leverage HCatalog to manage table schemas?

2.7 Other Ecosystem Tools

1. Give a sample Sqoop command to import a MySQL table orders into HDFS.
2. Describe how Flume agents ingest streaming log data into HDFS.
3. What workload characteristics make HBase preferable to plain HDFS storage?
4. Explain how Oozie orchestrates a multi-step Hadoop workflow.
5. Pick one additional tool (e.g., ZooKeeper, Kafka) and describe its role in the Hadoop world.

2.8 Where Hadoop Is Used in Data Engineering

1. List three types of pipelines or workloads where Hadoop still shines.
2. Why is Hadoop common for on-premises “data lake” architectures?
3. How does Hadoop integrate with Spark or Presto in modern ETL stacks?
4. What makes Hadoop less attractive for real-time or low-latency use cases?
5. Describe a scenario where legacy Hadoop clusters must coexist with newer cloud data platforms.

2.9 HDFS Architecture (Master-Slave & HA)

1. Describe the roles of the NameNode and DataNodes in HDFS.
2. What’s the function of the Secondary NameNode in Hadoop 1?
3. How does Hadoop 2+ achieve NameNode high availability?
4. Explain the pipeline write flow from client → DN1 → DN2 → DN3.
5. During a read, how does the client choose among multiple replicas?

2.10 MapReduce Deep Dive (Batch Engine)

1. Why is MapReduce considered a “batch” engine, and what overhead does that imply?
2. How do Combiners fit between Map and Reduce phases?
3. Explain how keys are sorted and shuffled before reaching reducers.
4. What is the role of the partitioner, and how can you customize it?
5. Discuss why MapReduce jobs may underutilize small-file or skewed-key datasets.

2.11 YARN Advanced (Speculative & Scheduling)

1. Define speculative execution and its impact on straggler tasks.
2. Compare CapacityScheduler vs. FairScheduler in YARN.
3. How does YARN handle multi-tenant resource isolation?
4. What metric does YARN use to kill or prioritize containers under memory pressure?
5. Describe how you would monitor and debug YARN resource allocation issues.

2.12 Memory Management & Block Size

1. Why must HDFS block sizes be powers of two?
2. What problems arise from very small blocks (e.g., 2 MB each)?
3. How do you change the default block size cluster-wide?
4. Explain a manual compaction strategy for small files.
5. When might you deliberately keep many small files instead of compacting them?

2.13 Erasure Coding (Hadoop 3+)

1. Compare 3× replication vs. 6+3 erasure coding in storage overhead and fault tolerance.
2. How many simultaneous block failures can an EC scheme with 6 data + 3 parity tolerate?
3. Why is EC better suited to “cold” or archival data?
4. What trade-offs come with enabling erasure coding on a directory?
5. How do you enable EC on an existing HDFS path?

2.14 HDFS Read & Write Anatomy

1. Walk through the client’s write path: NameNode lookup → block allocation → pipelined writes → acknowledgments.
2. During writes, what triggers the NameNode metadata update?
3. In the read path, how does the client leverage rack awareness?
4. What happens if a DataNode fails mid-write?

5. How does pipelining improve write throughput?

2.15 Hadoop CLI Commands

1. Which command lists `/user/data` contents in human-readable sizes?
2. Show the CLI to upload `local.csv` into HDFS `/user/ingest`.
3. How do you change replication of `/user/data/file.txt` to 2 via CLI?
4. Write the command to merge all files in `/logs` into a single local file `all.log`.
5. Which CLI subcommand runs a file-system health check?

2.15 Data Partitioning (Optimizing Reducer Load)

1. How does the number of partitions relate to reducers in MapReduce?
2. What is key skew and how does it affect reducer performance?
3. Give an example of writing a custom Partitioner in Java (pseudocode).
4. In Hive, how do you create and load a table partitioned by `event_date`?
5. Explain how bucketing complements partitioning for finer data pruning.

2.16 File Formats in Hadoop

1. Compare Parquet vs. ORC in terms of compression ratio and schema evolution.
2. Show a Spark (Scala or Python) snippet that reads an ORC file.
3. Why do columnar formats outperform CSV for analytic queries?
4. What is predicate pushdown, and how do Parquet/ORC support it?
5. When might you choose Avro over a columnar format in your pipeline?